

Supplementary Material for SMAT-God: A Multi-Agent Collaborative Framework for Detecting God Class in AI-Generated Code

Anonymous Authors

This document provides the complete prompt templates and machine-readable output structures used in the study. It is distributed separately from the paper so that the main manuscript can focus on the research design, empirical findings, and evaluation while retaining full reproducibility details.

1 Prompt Templates

This appendix presents the principal prompt templates used in our experiments. Runtime inputs are represented by placeholders enclosed in angle brackets. For example, `<SOURCE_CODE>` denotes the source code of the class under analysis. For the Decision Agent, all three evidence sources are enabled in the default configuration.

1.1 Code Description Generation

```
You are an expert software static analysis tool. Analyze the following code class and
    summarize the functionality of this class.

<SOURCE_CODE>

Just give the description, no explanation.
```

Listing 1: Code Description Generation Prompt

1.2 Code Reconstruction

```
You are an expert developer. Based on the technical description provided, reconstruct the
    corresponding code class.

Output only the source code. No explanation.

Description:
<CODE_DESCRIPTION>
```

Listing 2: Code Reconstruction Prompt

1.3 Semantic Agent

```
Role: Expert Software Architect

Task: Perform a deep semantic analysis of the provided class source code to evaluate its
      functional intent and responsibility structure.

Instructions:
1. Core Intent: Define the primary high-level purpose of this class in a single, clear
   sentence.
2. Responsibility Mapping: Provide a detailed breakdown of the distinct functional tasks
   found within the code. Identify whether the class mixes different business domains or
   abstraction layers.
3. Logical Depth & Flow: Analyze the complexity of the control flow. Describe why the code
   is dense or difficult to follow, such as complex state transitions or heavily branched
   business rules.
4. Design Critique: Provide a qualitative assessment of the class design. Determine whether
   it is focused or shows signs of becoming a "catch-all" container for unrelated logic.

Output Constraints:
- Output MUST be a valid JSON object.
- Use precise, professional English.
- Focus entirely on the meaning and intent of the code, rather than structural connectivity
  or static metrics.

JSON Schema:
{
  "class_name": "string",
  "core_intent": "string",
  "functional_responsibilities": [
    "Responsibility 1",
    "Responsibility 2"
  ],
  "logic_complexity_analysis":
    "Description of the logic depth and cognitive load",
  "semantic_design_assessment":
    "Critique focusing on responsibility balance"
}

Do not include any text outside the JSON object.
```

Listing 3: Semantic Agent System Prompt

1.4 Decision Agent

```
You are the Lead Architect and Final Arbiter in a code smell detection system.

Your task is to synthesize reports from three specialists:
- Semantic Agent: qualitative logic audit;
- Metric Agent: quantitative metric analysis;
- Responsibility Agent: semantic distance between methods.

Based on these reports, provide a final verdict: is this a "God Class"?
```

Listing 4: Decision Agent System Prompt

```

### [Source Code]
<SOURCE_CODE>

### [Expert Evidence Package]

[Metric Agent Report]
<METRIC_REPORT>

[Responsibility Agent Report]
<RESPONSIBILITY_GAP_REPORT>

[Semantic Agent Report]
<SEMANTIC_AUDIT_REPORT>

Analyze the consistency between the Expert Evidence and the Initial Human Label, and justify
the label based on the evidence.

```

Listing 5: Decision Agent User Prompt

1.5 Few-Shot Detection

```

You are a senior software architect specializing in code smell detection. Your job is to
decide whether a class is a God Class.

Judgment Rules:
1. judge_label = 1 means that the class centralizes too many responsibilities, mixes
   unrelated concerns, becomes a bloated coordinator, or exhibits low cohesion.
2. judge_label = 0 means that the class remains cohesive and focused on a limited
   responsibility.
3. Learn from the few-shot reference examples before judging the target class when such
   examples are provided.
4. Return only a JSON object with exactly these fields:
   - judge_label: 0 or 1
   - reasoning_process: a concise but clear explanation

The following reference examples were retrieved from the training set based on code-
embedding similarity.

### [Reference Examples]
<RETRIEVED_FIVE_EXAMPLES>

Analyze the following target class and determine whether it is a God Class.

### [Target Class]
<SOURCE_CODE>

```

Listing 6: Few-Shot Prompt

2 Sub-Agent Output Schemas

This appendix presents the output schemas of the three sub-agents used in SMAT-God. The concrete numerical values are omitted; only the structure and semantic meaning of each field are shown.

2.1 Metrics Agent Output

```
1 {
2   "agent_id": "string",
3   "target_class": "string",
4   "metric_values": {
5     "LOC": "number",
6     "WMC": "number",
7     "WMC/LOC": "number",
8     "LCOM4": "number",
9     "LCOM_CK": "number",
10    "CBO": "number",
11    "RFC": "number"
12  },
13  "statistics_report": {
14    "<metric_name>": {
15      "p0": "number",
16      "p10": "number",
17      "p20": "number",
18      "p30": "number",
19      "p40": "number",
20      "p50": "number",
21      "p60": "number",
22      "p70": "number",
23      "p80": "number",
24      "p90": "number",
25      "p100": "number"
26    }
27  }
28 }
```

Listing 7: Output schema of the Metrics Agent

Here, **metric_values** contains the metrics calculated for the target class. **statistics_report** provides percentile statistics for each metric, enabling the target class to be compared with the overall test distribution.

2.2 Responsibility Agent Output

```
1 {
2   "agent_id": "string",
3   "target_class": "string",
4   "results": {
5     "method_embedding_cosine_distance": {
6       "max": "number",
7       "min": "number",
8       "mean": "number"
9     },
10    "max_distance_method_pair": [
11      "method_name",
12      "method_name"
13    ],
14    "min_distance_method_pair": [
15      "method_name",
16      "method_name"
17    ]
18  }
19 }
```

```

17 ],
18 "statistics_report": {
19     "max": {
20         "p0": "number",
21         "p10": "number",
22         "...": "number",
23         "p100": "number"
24     },
25     "min": {
26         "p0": "number",
27         "p10": "number",
28         "...": "number",
29         "p100": "number"
30     },
31     "mean": {
32         "p0": "number",
33         "p10": "number",
34         "...": "number",
35         "p100": "number"
36     }
37 }
38 }
39 }

```

Listing 8: Output schema of the Responsibility Agent

The `method_embedding_cosine_distance` object summarizes the maximum, minimum, and mean semantic distances between method embeddings. `max_distance_method_pair` and `min_distance_method_pair` identify the method pairs associated with the largest and smallest responsibility differences, respectively. The nested `statistics_report` places these distance values in the corresponding dataset-level percentile distributions.

2.3 Semantic Agent Output

```

1 {
2     "class_name": "string",
3     "core_intent": "string",
4     "functional_responsibilities": [
5         "string"
6     ],
7     "logic_complexity_analysis": "string",
8     "semantic_design_assessment": "string",
9     "agent_id": "string"
10 }

```

Listing 9: Output schema of the Semantic Agent

The `class_name` field identifies the analyzed class. `core_intent` summarizes its main purpose. `functional_responsibilities` lists the distinct functions identified by the agent. `logic_complexity_analysis` describes the complexity of the control flow and state transitions, while `semantic_design_assessment` evaluates whether the class is cohesive or combines unrelated responsibilities. The `agent_id` field records the source of the analysis.